

HERMES A1

****Sistema Autónomo de Adquisición y Telemetría****

* ****MCU:**** STM32F446RET6

* ****RTOS:**** FreeRTOS

* ****Sensor:**** ADXL355

* ****Conectividad:**** Quectel EC25

* ****Versión:**** Documentación Técnica v1.1.0

* ****Autor:**** Antonio Avendaño Sanhueza

* ****Organización:**** LIND Engineering

* ****Fecha/Ubicación:**** Mayo 2026 Concepción, Chile

Especificaciones Técnicas

* ****MCU:**** STM32F446RET6. Cortex-M4 FPU, 512KB Flash, 128KB RAM.

* ****FRECUENCIA:**** 96 MHz. PLLCLK desde HSE.

* ****RTOS:**** FreeRTOS v10.3.1. CMSIS-RTOS v2 wrapper.

* ****PACKAGE:**** LQFP64. 64 pines.

* ****IDE:**** STM32CubeIDE. CubeMX v6.15.0 + GCC.

* ****HAL:**** STM32Cube FW_F4 V1.28.3.

* ****HEAP RTOS:**** 40 KB. ``configTOTAL_HEAP_SIZE = 40960``.

* ****TAREAS:**** 5 (4 activas + 1 idle). colas + eventos + mutex.

Índice

- **Visión General****
- **Hardware:**** sistema, hardware, evolución desde AWTAS.
- **Arquitectura de Software:**** pines, periféricos, NVIC, niveles eléctricos. tareas, colas, eventos, flujo de datos.
- **Gestión de Memoria:**** mapa de memoria, Heap_4, dimensionamiento de cola.
- **Estado DMA y FPU:**** configuración actual, impacto, recomendaciones.
- **Correcciones Aplicadas:**** 8 bugs identificados y resueltos.
- **Análisis Comparativo:**** 4 versiones del proyecto.
- **Evaluación UniKnect SDK:**** viabilidad técnica.
- **Actividades Pendientes:**** roadmap desde evaluación del prototipo.
- **Referencia API:**** documentación Doxygen.
- **Compilación y Flash****.
- **Comandos CLI****.
- **Estructura del Proyecto****.

1. Visión General

Firmware para STM32F446RET6 (Cortex-M4) del sistema HERMES A1. Adquisición autónoma de vibraciones y telemetría vía 4G/LTE. Desarrollado por Antonio Avendaño Sanhueza para LIND Engineering, Concepción, Chile.

1.1 Componentes del Sistema

Componente	Modelo	Interfaz	Función
MCU	STM32F446RET6	N/A	Procesamiento, control, RTOS
Acelerómetro	ADXL355	SPI2 (8 MHz)	Medición de vibración 3 ejes, 2/4/8g
Módem	Quectel EC25	USART1 (115200 baud)	Conectividad 4G/LTE, HTTP POST
Almacenamiento	microSD	SPI1 (8 MHz) + FatFs	Registro CSV de lecturas
Debug/CLI	N/A	USART2 (115200 baud)	Interfaz de comandos, diagnóstico

1.2 Principio de Operación

El sistema opera en modo triggered acquisition: el ADXL355 genera una interrupción (INT1) cuando la aceleración supera un umbral configurable. El firmware lee el FIFO del acelerómetro, almacena las lecturas en SD como archivos CSV, y las transmite a un servidor remoto vía HTTP/4G. El pipeline de datos es asíncrono y concurrente: la adquisición del sensor, la escritura en SD, la transmisión por módem y la interfaz de usuario operan como tareas independientes de FreeRTOS, coordinadas mediante colas y event flags.

1.3 Evolución: AWTAS → HERMES A1

Dimensión	AWTAS (predecesor)	HERMES A1 (actual)
Arquitectura	Baremetal, super-loop en main.c (~58 KB)	FreeRTOS + CMSIS-RTOS v2, 5 tareas
Concurrencia	Secuencial: sensor -> SD -> módem (bloqueante)	Concurrente: cola + event flags + mutex
CLI	Menú de 1 carácter (m, I, r, o, t, i, q)	Comandos completos (help, status, accel, trigger, sdtest)
Sincronización	Variables volátiles + busy-wait	osMessageQueue, osEvent Flags, osMutex
Tests	Ninguno	47 tests unitarios (Ceedling + Unity)

2. Hardware

2.1 Asignación de Periféricos

Periférico	Pines	Modo	Velocidad	Target	Nivel Lógico
SPI1	PA5, PA6, PA7, PB6(CS)	Full-Duplex Master	8.0 MB/s	microSD	3.3V
SPI2	PB13, PB14, PB15, PA4(CS)	Full-Duplex Master	8.0 MB/s	ADXL355	3.3V
USART1	PA9(TX), PA10(RX)	Asíncrono	115200 baud	Quectel EC25	1.8V (módem)
USART2	PA2(TX), PA3(RX)	Asíncrono	115200 baud	CLI/Debug	3.3V

Datos validados desde CubeMX.ioc. Velocidades SPI y configuraciones validadas desde HERMES-A1-CMSIS.ioc (CubeMX v6.15.0). Frecuencia del sistema: 96 MHz (PLLCLK).

2.2 Pinout Completo

Pin	Etiqueta	Función	Configuración	Destino
SPI1 **microSD**				
PA5	SPI1_SCK	Clock	AF5, Push-Pull	SD CLK
PA6	SPI1_MISO	Data In	AF5	SD MISO
PA7	SPI1_MOSI	Data Out	AF5	SD MOSI
PB6	SD_CS	Chip Select	Output PP, High, Speed HIGH	SD CS
SPI2 **ADXL355**				
PA4	ADXL_CS	Chip Select	Output PP, High, Speed HIGH	ADXL355 CS
PB13	SPI2_SCK	Clock	AF5	ADXL355 SCLK
PB14	SPI2_MISO	Data In	AF5	ADXL355 MISO
PB15	SPI2_MOSI	Data Out	AF5	ADXL355 MOSI
USART1 **Quectel EC25**				
PA9	USART1_TX	Transmit	AF7	EC25 RX
PA10	USART1_RX	Receive	AF7	EC25 TX
USART2 **CLI / Debug**				
PA2	USART2_TX	Transmit	AF7	USB-serial RX
PA3	USART2_RX	Receive	AF7	USB-serial TX
GPIO **Control Módem**				
PB0	HAT_PWR_OFF	Apagado EC25	Output PP, Low, Pull Down	EC25 PWR_OFF
PB1	MODEM_PWRKEY	Encendido EC25	Output PP, Low, Pull Down	EC25 PWRKEY
PB2	MODEM_RI	Ring Indicator	Input EXTI Falling, Pull Up	EC25 RI
GPIO **ADXL355**				
PC0	ADXL_DRDY	Data Ready	Input	ADXL355 DRDY
PC7	ADXL_INT1	Interrupción	Input EXTI (prio 5), Pull Down	ADXL355 INT1
Depuración **(SWD)**				
PA13	SYS_JTMS-SWDIO	SWD Data	Alternate	ST-Link
PA14	SYS_JTCK-SWCLK	SWD Clock	Alternate	ST-Link
PB3	SYS_JTDO-SW0	SW0 Trace	Alternate	ST-Link

2.3 Niveles Eléctricos y Tolerancias

Señal	Nivel MCU	Nivel Target	Notas
SPI1 (SD)	3.3V (VDD)	3.3V	Compatible directo. SD CS: Push-Pull, speed HIGH
SPI2 (ADXL355)	3.3V (VDD)	3.3V (VDD del ADXL355)	Compatible directo. ADXL_CS: Push-Pull, speed HIGH
USART1 (EC25)	3.3V (VDD)	1.8V (nivel módem)	Requiere level shifter o resistencias serie
USART2 (CLI)	3.3V (VDD)	3.3V (USB-serial)	Compatible directo
ADXL_INT1	3.3V (tolerante 5V)	3.3V	EXTI falling edge, pull-down interno
MODEM_RI	3.3V (tolerante 5V)	1.8V-2.8V (EC25)	EXTI falling edge, pull-up interno

ATENCIÓN: USART1 opera a 3.3V en el STM32 pero el EC25 espera 1.8V en sus pines UART. Verificar que el hardware incluye level shifting o resistencias serie de adaptación.

2.4 Configuración NVIC

IRQ	Fuente	Preempt Priority	Sub Priority	Propósito
:---	:---	:---	:---	:---
EXTI9_5_IRQn	PC7 (ADXL_INT1)	5	0	Detección de movimiento (motion trigger)
TIM1_UP_TIM10_IRQn	TIM1	15	0	Base de tiempo del RTOS (SysTick sustituto)
PendSV_IRQn	FreeRTOS	15	0	Context switch del scheduler
SysTick_IRQn	SysTick	15	0	Timebase HAL

3. Arquitectura de Software

3.1 Tareas FreeRTOS

Tarea	Función	Prioridad CMSIS	Stack (words)	Stack (bytes)	Entrada
:---	:---	:---	:---	:---	:---
sensor_task	StartSensorTask	osPriorityHigh (40)	256	1024	EXTI (ADXL_INT1)
modem_task	StartModemTask	osPriorityAboveNormal (32)	512	2048	EVT_FILE_READY
file_task	StartFileTask	osPriorityNormal (24)	256	1024	sensor_queue
control_task	StartControlTask	osPriorityNormal (24)	1024	4096	UART2 (CLI)
defaultTask	StartDefaultTask	osPriorityNormal (24)	128	512	(idle loop)

Nota sobre prioridades numéricas: CMSIS-RTOS v2 mapea prioridades sobre un rango de 56 niveles (configMAX_PRIORITIES). Los valores entre paréntesis son la prioridad numérica interna. Mayor número = mayor prioridad.

3.2 Objetos de Sincronización

Objeto	Tipo	Configuración	Uso
:---	:---	:---	:---
sensor_queue	osMessageQueue	50 slots x 28 bytes	sensor_task → file_task (lecturas)
sd_mutex	osMutex	Recursive	Protección de acceso a SD (FatFs)
sensor_event_flags	osEventFlags	5 flags	Coordinación entre tareas

3.3 Event Flags

Flag	Bit	Productor	Consumidor	Semántica	
:---	:---	:---	:---	:---	
EVT_MOTION_DETECTED	(1<<0)	EXTI ISR/	control_task	sensor_task	Iniciar adquisición
EVT_ACQSTN_DONE	(1<<1)	sensor_task	file_task		Adquisición completada
EVT_UPLOAD_DONE	(1<<2)	modem_task	N/A		Upload completado (monitor)
EVT_CFG_CHECK	(1<<3)	Timer periódico	control_task		Verificar configuración remota
EVT_FILE_READY	(1<<4)	file_task	modem_task		Archivo CSV listo para upload

3.4 Flujo de Datos

1. ``ADXL_INT1 (EXTI)`` → ``sensor_task``
2. ``sensor_queue`` (50 x 28 bytes) → ``file_task`` (Escribe CSV en SD)
3. ``EVT_ACQSTN_DONE`` (Señal de fin de adquisición)
4. ``EVT_FILE_READY`` → ``modem_task`` (Ejecuta HTTP POST al backend)
5. ``control_task`` recibe ``UART2 (CLI)`` para comandos de usuario.

3.5 Tipo de Dato SensorReading_t

```

``c
typedef struct {
    uint32_t timestamp_ms; // 4 bytes ms desde boot
    float x_g; // 4 bytes aceleración X
    float y_g; // 4 bytes aceleración Y
    float z_g; // 4 bytes aceleración Z
    float voltage; // 4 bytes reservado (siempre 0.0f)
    float current; // 4 bytes reservado (siempre 0.0f)
    float power; // 4 bytes reservado (siempre 0.0f)
    // Total: 28 bytes
} SensorReading_t;

```

Nota: Los campos voltage, current y power están reservados para telemetría energética (pendiente §1.5 del roadmap). Actualmente siempre contienen 0.0f.

3.6 Lógica de sensor_task

La tarea de sensor opera en un ciclo de estados implícito:

1. **Espera:** `osEventFlagsWait(EVT_MOTION_DETECTED, timeout=5000ms)` heartbeat cada 5s.
2. **Adquisición:** Lee ADXL355 cada 10ms (10 Hz), encola cada muestra en `sensor_queue`.
3. **Settling detection:** Si la magnitud XY cae bajo `trigger_g` por 3s consecutivos ($\Delta < 0.005g$), termina la adquisición.
4. **Earthquake rejection:** Si cualquier eje supera $\pm 2g$, aborta la adquisición (protección contra saturación).
5. **Notificación:** Sube `EVT_ACQSTN_DONE`, `file_task` cierra archivo, sube `EVT_FILE_READY`, arranca `modem_task`.

Duración máxima de adquisición: 15 minutos (900,000 ms). Período de muestreo: 10ms (10 Hz) vía `osDelay(10)`.

4. Gestión de Memoria

4.1 Mapa de Memoria del STM32F446RE

Región	Dirección	Tamaño	Uso
Flash	0x0800 0000	512 KB	Código + constantes
SRAM	0x2000 0000	128 KB	Variables globales + heap + stacks
CCM RAM	0x1000 0000	64 KB	No utilizado (podría optimizarse)

4.2 Distribución del Heap FreeRTOS (Heap_4)

`configTOTAL_HEAP_SIZE = 40960` (40 KB) de los 128 KB de SRAM disponibles.

Componente	Tamaño	Cálculo
sensor_task stack	1024 B	256 words x 4 bytes
modem_task stack	2048 B	512 words x 4 bytes
file_task stack	1024 B	256 words x 4 bytes
control_task stack	4096 B	1024 words x 4 bytes
default Task stack	512 B	128 words x 4 bytes
sensor_queue	1400 B	50 slots x 28 bytes
Queue overhead	~200 B	Structura interna FreeRTOS
Mutex + EventFlags	~200 B	Structuras internas
Total asignado	~10,500 B	~26% del heap de 40 KB
Heap libre	~29,500 B	Disponible para malloc dinámico

4.3 Justificación de Heap_4

FreeRTOS ofrece 5 esquemas de heap. Se seleccionó Heap_4 por:

- **Coalescencia:** fusiona bloques libres adyacentes, reduciendo la fragmentación externa.
- **Alloc + free:** soporta liberación segura (a diferencia de Heap_1 o Heap_2).
- **First-fit:** Algoritmo determinista siempre busca desde el inicio del heap.

Riesgo residual: En operación continua (24/7), la fragmentación puede acumularse.

Mitigación: usar `xPortGetFreeHeapSize()` periódicamente para monitorear.

4.4 Dimensionamiento de sensor_queue

La cola tiene 50 slots x 28 bytes = 1400 bytes. El dimensionamiento se deriva de:

`N_slots >= (f_sample x t_write_max) + margen_de_seguridad`

- `f_sample` = 10 Hz
- `margen_de_seguridad` = 2x para absorber picos de latencia.

Si `file_task` tarda 500ms en escribir un bloque a SD (latencia alta de FAT):

`N_slots = (10 muestras/s x 0.5s) x 2 = 10 slots`.

50 slots proporciona 5x margen (5 segundos de adquisición bufferizada a 10 Hz).

5. Estado DMA y FPU

5.1 FPU Habilitada (Crítico) IMPLEMENTADO

Parámetro	Valor Actual	Recomendado
configENABLE_FPU	1 (habilitada)	1 (habilitada)

Impacto: `sensor_task` ejecuta operaciones float por hardware (~1-2 ciclos/operación). A 10 Hz con 3-5 operaciones float por muestra, el overhead es de ~30-50 ciclos/s. El Cortex-M4 tiene FPU hardware.

CRÍTICO: Habilitar FPU (`configENABLE_FPU = 1`) en `FreeRTOSConfig.h` antes de aumentar la tasa de muestreo o agregar procesamiento de señal (FFT, RMS, SNR).

Estado:  Completado - FPU habilitada en `FreeRTOSConfig.h` línea 59. Validado con `test_fpu` en CLI y test unitario `test_fpu_operations()` .

5.2 DMA (Configurado) IMPLEMENTADO

Interfaz	Método Actual	DMA Configurado	Impacto
SPI1 (SD)	Polling (<code>USE_DMA = 0</code>)	No	CPU bloqueada durante transferencias SPI
SPI2 (ADXL355)	DMA (<code>HAL_SPI_TransmitReceive_DMA</code>)	Sí (DMA1 Stream 3/4, Ch 0)	Transferencia continua sin bloquear CPU
USART1 (EC25)	Polling (<code>HAL_UART_Transmit/Receive</code>)	No	CPU bloqueada durante comandos AT
USART2 (CLI)	Polling (<code>printf</code> → UART)	No	Menor impacto (solo debug)

Impacto en concurrencia: Con DMA para SPI2, la CPU puede ejecutar otras tareas mientras el acelerómetro transfiere datos. Una lectura de FIFO ADXL355 ya no bloquea la CPU.

Estado:  Completado - DMA SPI2 implementado en `adxl355.c` (líneas 418-548):

- DMA1 Stream 3 (RX) + Stream 4 (TX), Channel 0
- Semáforo binario para sincronización task↔ISR
- Función `ADXL355_Read_Data_DMA()` para lectura full-duplex
- ISRs: `DMA1_Stream3_IRQHandler` , `DMA1_Stream4_IRQHandler`
- Callback: `HAL_SPI_TxRxCpltCallback` libera semáforo

Validado con comando CLI `dma` y test unitario `test_spi2_dma()` .

5.3 Wake-on-Motion (WoM) Interrupt IMPLEMENTADO

Parámetro	Valor Actual	Estado
INT1 (PC7)	EXTI Falling Edge, Pull-Down	 Configurado
Activity Detection	X+Y axes, HPF OFF	 Configurado
Threshold	Configurable via <code>ADXL355_Config_WakeOnMotion()</code>	 Implementado
CLI Test	Comando <code>wom</code>	 Disponible

Estado:  Completado - WoM implementado en `adxl355.c` (líneas 235-270):

- Función `ADXL355_Config_WakeOnMotion(threshold_g, count)`
- Umbral configurable en G, cuenta de muestras consecutivas
- Ejes X+Y habilitados (evita falsos positivos por gravedad en Z)
- Interrupción mapeada a INT1 (PC7, EXTI9_5_IRQn, prioridad 5)
- Validado con comando CLI `wom` y test unitario `test_wom_interrupt()`

6. Correcciones de Código Aplicadas

#	Problema	Archivo	Solución
1	<code>strcasecmp()</code> no disponible en ARM GCC	<code>main.c</code>	Implementación local <code>strcasecmp_custom()</code>

#	Problema	Archivo	Solución
2	Inclúdes con mayúsculas incorrectas	6 archivos	Corregido a minúsculas consistentes
3	Escrituras a registros EXTI sin deshabilitar IRQ	control_task.c	__disable_irq() / __enable_irq() alrededor de EXTI
4	Falso positivo EVT_ACQSTN_DONE en sdtest	sensor_task.c	Guard !sdbg_abort_acq antes de osEventFlagsSet
5	Variable muerta FRESULT fres	main.c	Eliminada, llamada directa a sd_mount()
6	Credenciales vacías sin validación	credentials.h	Función Credentials_Validate() inline
7	FIFO ADXL355 leído con CS low/high por muestra	adxl355.c	Burst read: CS bajo una vez, leer todas las muestras
8	Llamadas al módem antes de osKernelStart()	main.c	Comentadas; mover a modem_task

7. Análisis Comparativo de Versiones

Proyecto	Ubicación	Arquitectura	Estado
cmsis-test (local)	Directorio actual	FreeRTOS, 4 tareas + algo/, 47 tests	Versión de trabajo
cmsis-test (GitHub)	github.com/tonetooo/cmsis-test	FreeRTOS, 4 tareas, sin algo/	Semi-estable
cmsis (GitHub)	github.com/tonetooo/cmsis	FreeRTOS, 4 tareas, sin test suite	Versión limpia
AWTAS	github.com/tonetooo/AWTAS	Baremetal, superloop, state machines	Predecesor

7.1 Diferencias Clave

Feature	Local	cmsis-test (GH)	cmsis (GH)	AWTAS
Archivos .c/.h	16/19	14/17	13/16	10/10
Directorio algo/	Sí	No	No	No
Tests unitarios	47	No	No	No
test_suite.c	Sí	Sí	No	No
RTOS	FreeRTOS 40KB	FreeRTOS	FreeRTOS	Baremetal
quectel_drive.c	866 líneas	~800	~800	744 líneas

8. Evaluación de Quectel UniKnect SDK

Componente	UniKnect SDK	Nuestro Proyecto	Veredicto
MCU	STM32F413RGT6/VGT6	STM32F446RET6	No compatible
Módem	EC200U, BG96	Quectel EC25	AT similar, no verificado
Build	CMake	CubeMX Makefile	Requiere migración
RTOS	FreeRTOS bundled	FreeRTOS + CMSIS-RTOS v2	Posible conflicto

ATENCIÓN: No recomendado para la versión actual. El costo de portar supera los beneficios.

9. Actividades Pendientes - Roadmap

9.1 Prioridad 1 - Crítico (operación confiable en terreno)

#	Actividad	Implementación	Archivos	Estado
9.1.1	DMA para adquisición de datos	Configurar DMA1 Stream 3/4 (RX/TX), Channel 0, modo normal	adxl355.c (líneas 418-548), stm32f4xx_hal_msp.c, CubeMX.ioc	✅ COMPLETADO
9.1.2	Habilitar FPU	configENABLE_FPU = 1 en FreeRTOSConfig.h	FreeRTOSConfig.h (línea 59)	✅ COMPLETADO

#	Actividad	Implementación	Archivos	Estado
9.1.3	Adquisición en tiempo continuo	Modo CONTINUOUS en sensor_task	sensor_task.c, control_task.c	 EN PROGRESO (20%)
9.1.4	Calibración del acelerómetro	Comando CLI calibrate + coeficientes en flash	adxl355.c, control_task.c	 INICIADO (10%)
9.1.5	Watchdog	CubeMX IWDG + HAL_IWDG_Refresh()	CubeMX.ioc, freertos.c	 PENDIENTE
9.1.6	Buffer local ante pérdida de conectividad	Cola de archivos pendientes + reintento en modem_task	file_task.c, modem_task.c	 INICIADO (10%)
9.1.7	Sincronización horaria	NTP vía módem (AT+QNTTP)	quectel_drive.c, sensor_task.c	 INICIADO (10%)
9.1.8	Optimización de consumo energético	HAL_SuspendTick() en idle, apagar periféricos, reducir ODR	freertos.c, CubeMX.ioc	 PENDIENTE

9.2 Prioridad 2 - Alto (confiabilidad y diagnóstico remoto)

#	Actividad	Implementación	Archivos
9.2.1	Métricas de calidad de señal	AT+CSQ para RSSI, cálculo RMS	sensor_task.c, quectel_drive.c
9.2.2	Telemetría energética integrada	ADC1 para medir voltaje/corriente	sensor_task.c, stm32f4xx_hal_msp.c
9.2.3	Gestión remota de configuración	Parser + recarga runtime de config	quectel_drive.c
9.2.4	Sistema de alarmas operativas	HTTP POST de alerta a endpoint	sensor_task.c, modem_task.c
9.2.5	Monitoreo del estado interno	Comando CLI diag + CSV separado	control_task.c
9.2.6	Compensación térmica	Leer registro TEMPERATURE y aplicar corrección	adxl355.c, sensor_algo.c
9.2.7	Caracterización del enlace celular	Evaluar AT commands, red de adaptación de antena	quectel_drive.c

9.3 Prioridad 3 - Evolución (escalabilidad)

#	Actividad	Implementación	Archivos
9.3.1	Arquitectura multisensor I2C	Configurar I2C, TCA9548A, refactorizar adxl355.c	adxl355.c, CubeMX.ioc, tca9548a.c
9.3.2	Reinicio remoto	Endpoint HTTP que dispare NVIC_SystemReset()	quectel_drive.c, modem_task.c
9.3.3	Migración del protocolo de transmisión	Cliente FTP (AT+QFTP CMD) en EC25	quectel_drive.c

9.4 Matriz de Prioridades Resumida

- P1 (Crítico): DMA para SPI2, Habilitar FPU, Adquisición continua, Calibración acelerómetro, Watchdog, Buffer conectividad, Sincronización horaria, Optimización consumo.
- P2 (Alto): Métricas calidad señal, Telemetría energética, Gestión remota config, Alarmas operativas, Monitoreo interno, Compensación térmica, Caracterización celular.
- P3 (Bajo): Multisensor I2C, Reinicio remoto, Migración FTP/FTPS.

10. Referencia API

Salida Doxygen HTML: docs/doxygen/html/index.html

Módulo	Archivos	Responsabilidad
Driver ADXL355	adxl355.c/h	SPI2, lectura FIFO, conversión raw g, LevelToZero
Driver Quectel EC25	quectel_drive.c/h	PowerOn, BringUpNetwork, SendAT, UploadFile, DownloadConfig
Driver SD	sd_spi.c/h	SPI1, bloques 512B, FatFs diskio
Tareas RTOS	tasks/*.c	sensor, modem, file, control pipeline de datos
Algoritmos	algo/*.c/h	Lógica pura: motion detection, settling, CLI parser, CSV format
CLI	control_task.c	Parser de comandos, ejecución, output por UART2

Módulo	Archivos	Responsabilidad
Tests	test_suite.c/h	Diagnóstico: SD test, modem test, sensor test

11. Compilación y Flash

Compilar:

```
make -j4
```

Flashear:

```
openocd -f interface/stlink.cfg -f target/stm32f4x.cfg -c "program build/HERMES-A1-CMSIS.elf verify reset exit"
```

Conexión CLI:

```
picocom -b 115200 /dev/tty.usbmodem*
```

```
O screen /dev/tty.usbmodem* 115200
```

Tests Unitarios:

```
cd test && ceedling test:all (47 tests)
```

12. Comandos CLI

Comando	Descripción
help	Lista de comandos disponibles
status	Estado del sistema y umbral actual
accel	Lectura actual del acelerómetro
trigger	Umbral de disparo actual en G
trigger <valor>	Establecer umbral (0-10 G)
log	Lista archivos CSV en SD
test	Simula evento de movimiento (prueba de pipeline)
sdtest	Adquisición forzada 10s + tabla ASCII en SD
modem_on	Enciende módem + prueba sincronización AT
sensstat	Lee registro STATUS del ADXL355 (ACT/DRDY bits)
readreg <hex>	Lee cualquier registro ADXL355 (ej. readreg 0x2C)
at <cmd>	Envía comando AT crudo al módem (ej. at AT+CPIN?)
debug	Alterna salida diagnóstica (on/off)
fpu	Test operaciones FPU (hardware float)
dma	Test transferencia SPI2 DMA
wom	Test interrupción Wake-on-Motion

13. Estructura del Proyecto

Ruta	Contenido
Core/Inc/	Headers: main.h, adxl355.h, quectel_drive.h, sd_spi.h, tasks.h, etc.
Core/Inc/algo/	Lógica pura (test host): sensor_algo.h, cli_algo.h, csv_algo.h
Core/Src/	Implementación: main.c, adxl355.c, quectel_drive.c, sd_spi.c, etc.
Core/Src/algo/	Lógica pura: sensor_algo.c, cli_algo.c, csv_algo.c
Core/Src/tasks/	Tareas RTOS: sensor_task.c, file_task.c, modem_task.c, control_task.c
Core/Startup/	Startup assembly + linker script
Drivers/	STM32 HAL + CMSIS
FATFS/	FatFs middleware
Middlewares/	FreeRTOS + FatFs

Ruta	Contenido
Lib/FreeRTOS-LTS/	FreeRTOS LTS submodule
test/	Ceedling + Unity: 47 tests
docs/es/	Documentación español: HTML + PDF
docs/doxygen/	API reference generada por Doxygen
Doxyfile	Configuración Doxygen
Makefile	Build system (arm-none-eabi-gcc)
HERMES-A1-CMSIS.ioc	STM32CubeMX project config
STM32F446RETX_FLASH.ld	Linker script

